

Wirtualny Dziennik Szkolny

Autorzy projektu:

1. Dawid Ugniewski
2. Adam Babiński
3. Aleksander Józef Żyliński

Grupa PS: 7

Data: 14.11.2023

1. Wstęp.

Elektroniczny dziennik szkolny z możliwością przeprowadzania testów. Aplikacja umożliwia zarządzanie ocenami i przedmiotami oraz ich przeglądanie zarówno przez uczniów jak i przez rodziców.

2. Analiza wymagań systemu.

2.1. Wymagania funkcjonalne.

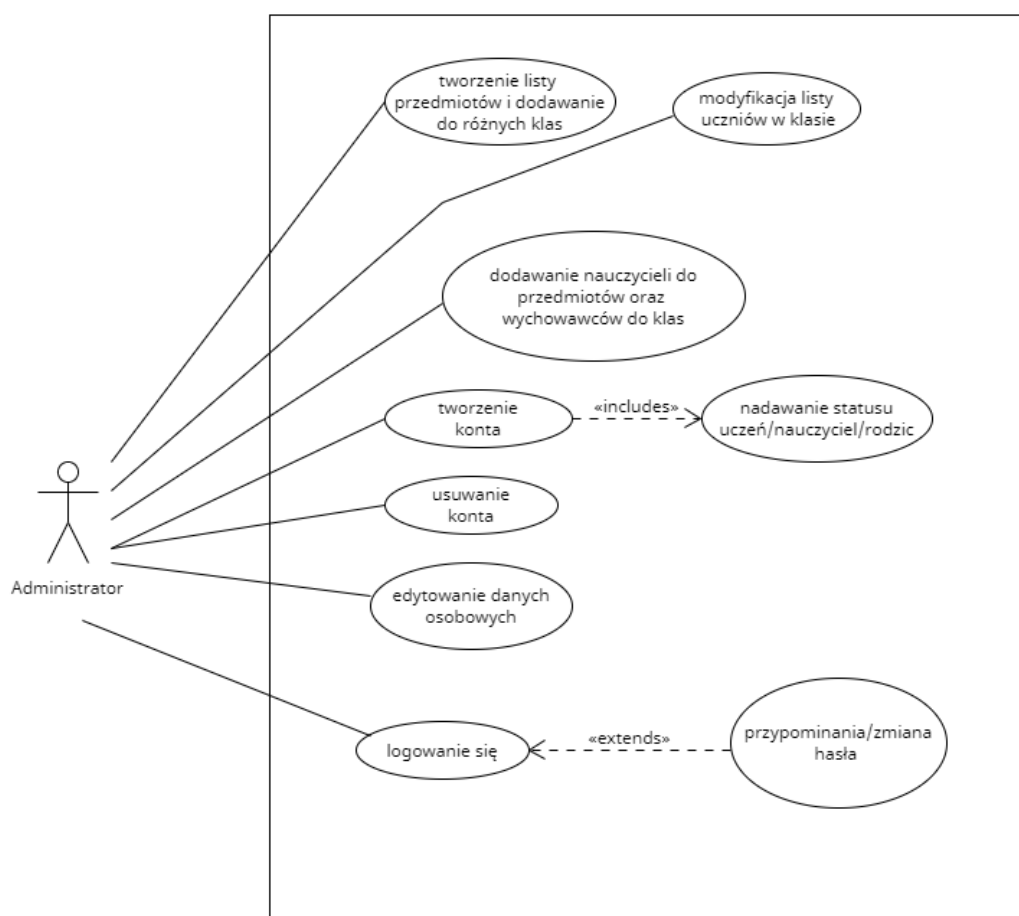
- modyfikacja listy uczniów należących do poszczególnych klas
- tworzenie listy przedmiotów oraz przyporządkowywanie ich do różnych klas
- dodawanie nauczycieli do przedmiotów oraz wychowawców dla klas
- możliwość edycji danych osobowych uczniów, nauczycieli oraz rodziców
- konta nauczycieli, uczniów oraz rodziców
- wystawianie oceny z przedmiotu przez nauczyciela
- wystawie oceny z zachowania przez wychowawcę
- przeglądanie zestawień ocen z zadanego okresu dla wybranych uczniów oraz przedmiotów
- okresowe wysyłanie zestawień ocen uczniów na konto mailowe rodziców

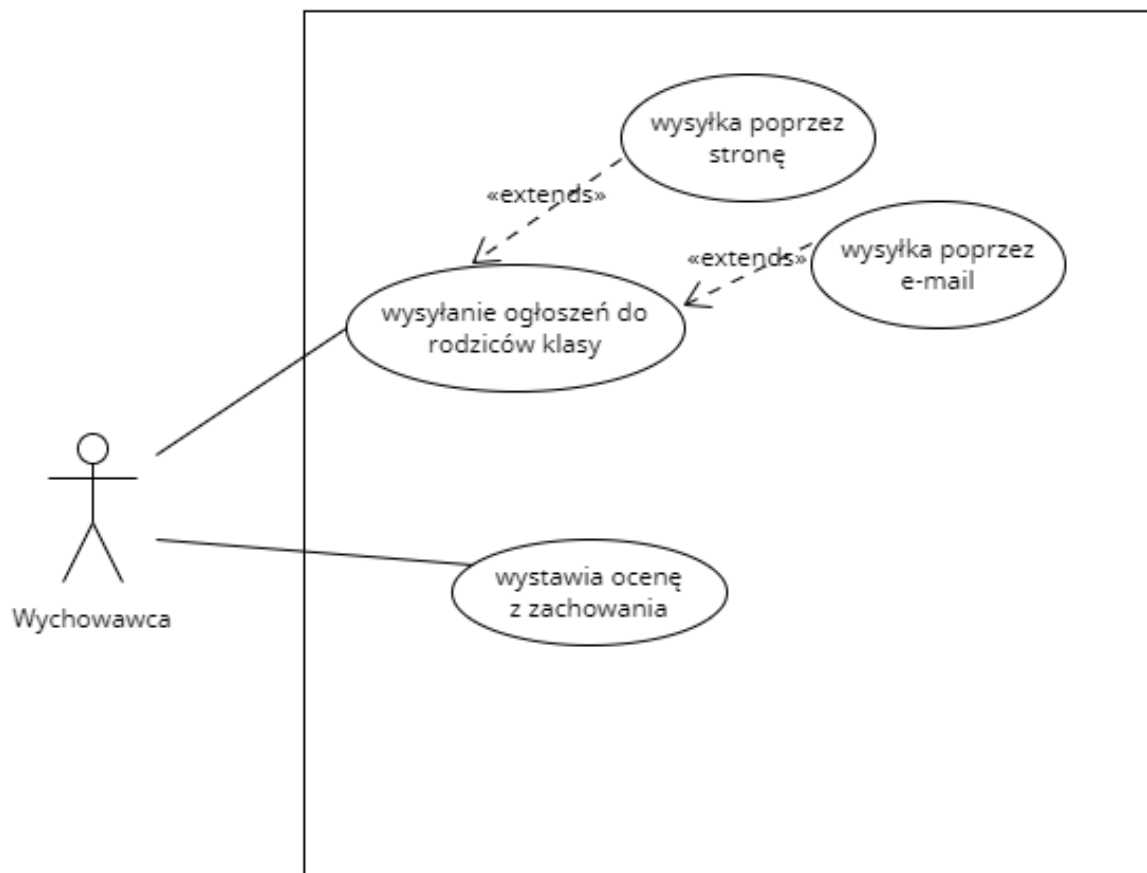
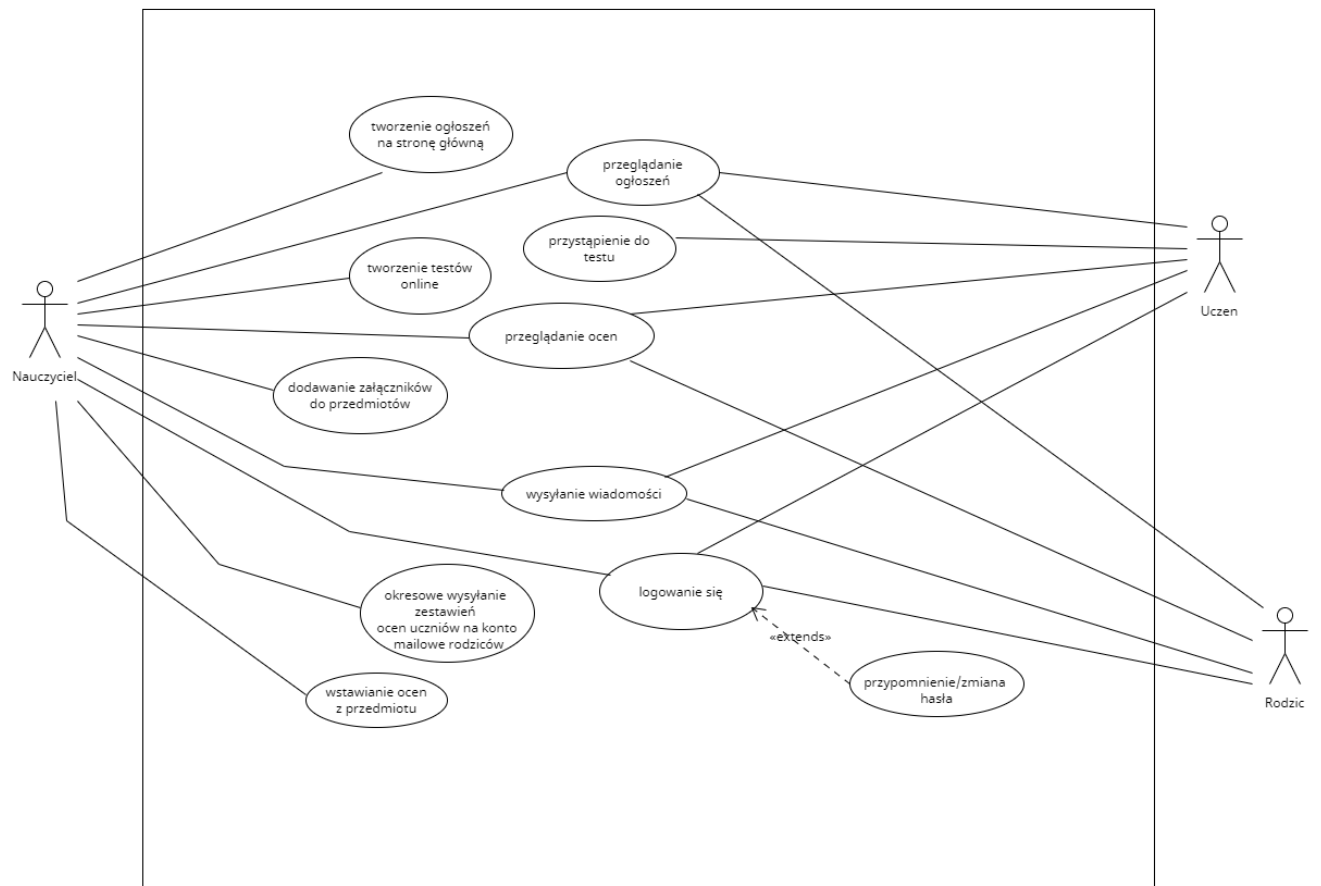
- komunikacja między nauczycielami a rodzicami oraz między nauczycielami a uczniami poprzez wiadomości
- wysyłanie ogłoszeń przez wychowawców do rodziców danej klasy zarówno przez email, jak i przez stronę
- lista ogłoszeń na głównej stronie serwisu
- możliwość dodawania plików przez nauczycieli do poszczególnych przedmiotów
- moduł obsługi pytań testowych definiowanych przez nauczyciela. Określenie czasu trwania testu, treści pytań oraz odpowiedzi, ilości punktów za poszczególne pytania. Ocena wystawiana automatycznie po zakończeniu testu
- bezpieczna procedura przypominania i zmiany hasła

2.2. Wymagania niefunkcjonalne.

- Automatycznie powiązuje w bazie nauczycieli, rodziców, uczniów i administratora z kontem.
- Wielkość załączników do ~16,77 MB.
- Hasła w bazie danych przechowywane w postaci skrótów do nich, obliczane w aplikacji.

2.3. Diagram przypadków użycia.





3. Wykorzystane technologie.

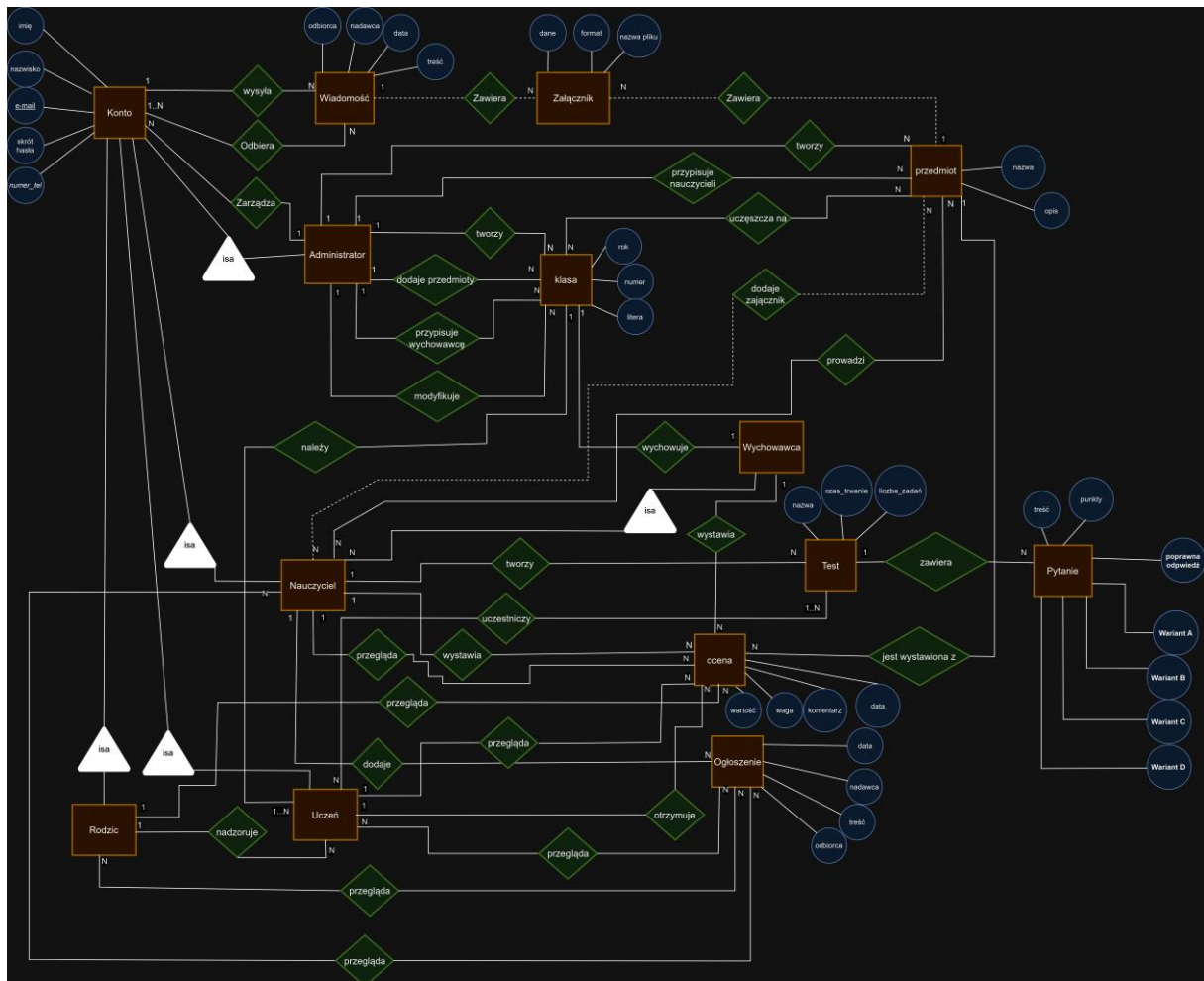
.NET, Entity Framework

4. Projekt aplikacji.

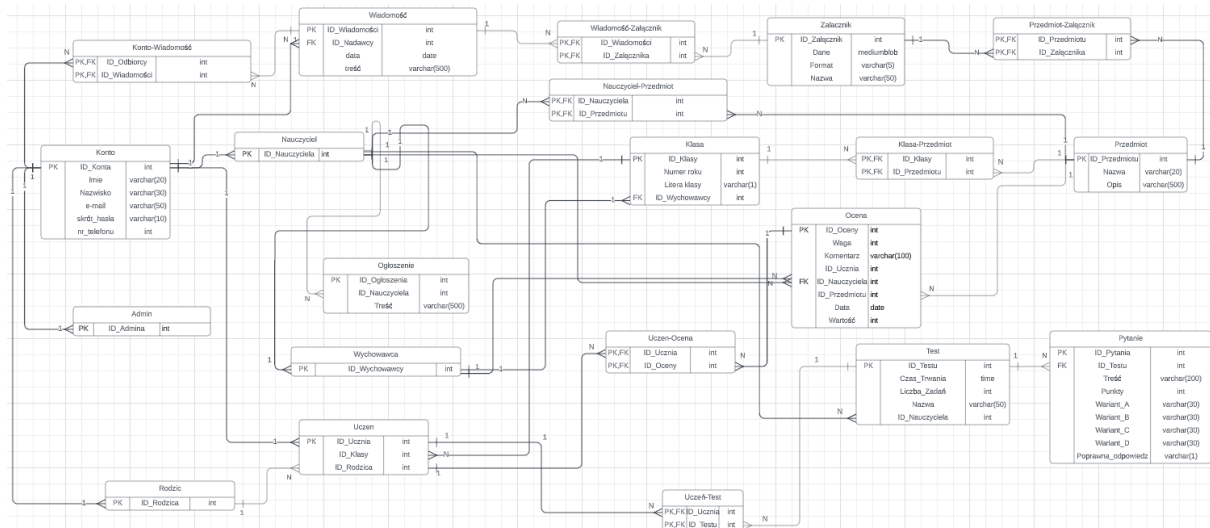
4.1. Architektura aplikacji.

Architektura MVC z odwzorowaniem ORM (Entity Framework) w relacyjnej bazie danych MySQL.

4.2. Projekt koncepcyjny bazy danych.



4.3. Projekt schematu relacyjnego:



Kod tworzący wszystkie tabele zawarte na diagramie znajduje się w pliku "implementacja_bazy.sql".
Serwerem bazodanowym jest MySQL.

- Konto-Wiadomość - przechowuje powiązania między wiadomościami a kontami, które są ich odbiorcami
- Wiadomość - przechowuje wiadomość, jej treść i nadawcę
- Wiadomość-Załącznik - przechowuje powiązania między załącznikami a wiadomościami, do których są załączone
- Załącznik - przechowuje załącznik, jego nazwę pliku, rozszerzenie i dane w postaci binarnej do 16,7 MB
- Przedmiot-Załącznik - przechowuje powiązania między załącznikami a przedmiotami, do których są załączone
- Nauczyciel – przechowuje nauczyciela
- Przedmiot – przechowuje przedmiot
- Nauczyciel-Przedmiot – przechowuje powiązania między nauczycielami a przedmiotami, których uczą
- Konto – przechowuje konto, dane personalne osoby oraz dane logowania
- Klasa – przechowuje klasę i informację o wychowawcy
- Klasa-Przedmiot – przechowuje powiązania między klasami, a przedmiotami na które uczęszczają
- Pytanie – przechowuje treść pytania testowego, punkty za dane pytanie/zadanie, warianty odpowiedzi od A do D oraz poprawną odpowiedź
- Test – przechowuje czas trwania testu, liczbę zadań testowych, nazwę testu oraz ID nauczyciela, który stworzył test
- Uczeń-Test – przechowuje powiązania między uczniami, a testami które rozwiązują
- Uczeń - przechowuje ucznia, klasę oraz rodzica
- Rodzic – przechowuje rodzica

q) Ocena – przechowuje wagę oceny, komentarz informujący z czego ona jest (sprawdzian, kartkówka, zachowanie etc.), przechowuje również ucznia oraz przedmiot, datę kiedy ocena została wstawiona, wartość oceny od 1 do 6

r) Uczeń-Ocena – przechowuje powiązania między uczniami, a ocenami jakie otrzymali/mają

s) Wychowawca – przechowuje wychowawcę

t) Admin – przechowuje admina

w) Ogłoszenia - przechowuje ogłoszenia, nauczyciela który jest autorem oraz treść ogłoszenia

4.4. Mapowanie klas na tabele bazodanowe

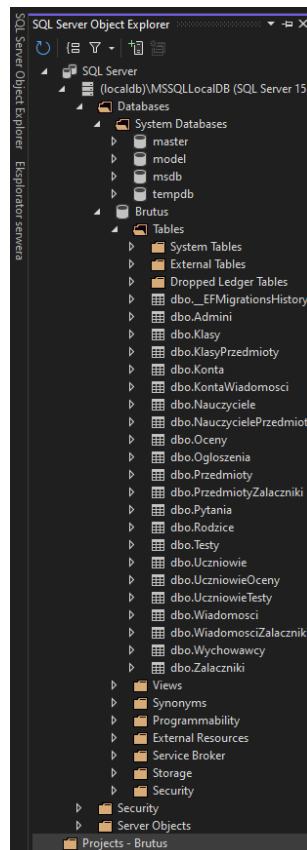
W folderze Brutus\Models znajdują się klasy odpowiadające tabelom bazy danych, napisane w C#, te klasy odpowiadają poszczególnym tabelom w naszym systemie. Wykorzystano funkcjonalności Entity Framework i system bazodanowy MySQL. Plik Brutus\Persistence\DzienniczekContext.cs zawiera klasę kontekstu bazy danych, umożliwiającą tworzenie bazy danych w modelu Code First i korzystanie z jej tabel za pomocą właściwości typu DbSet. To możliwe dzięki adnotacjom „Key” i „ForeignKey” oraz właściwościom nawigacyjnym w klasach modeli.

Migracja tworząca tabele znajduje się w ścieżce Brutus\Migrations\[...]InitialCreate.cs. Zostały one utworzone w terminalu za pomocą następujących poleceń:

```
dotnet ef migrations add InitialCreate
```

```
Add-Migration InitialCreate
```

```
Update-Database
```



Przykładowa tabela "Klasy":

dbo.Klasy [Design] | Script File: dbo.Klasy.sql

Name	Data Type	Allow Nulls	Default
ID_Klasy	int	☐	
NumerRoku	int	☐	
LiteraKlasy	nvarchar(1)	☐	
ID_Wychowawcy	int	☒	

Keys (1)
PK_Klasy (Primary Key, Clustered: ID_Klasy)

Check Constraints (0)

Indexes (1)
IX_Klasy_ID_Wychowawcy (ID_Wychowawcy)

Foreign Keys (1)
FK_Klasy_Wychowawcy_ID_Wychowawcy (ID_Wychowawcy)

Triggers (0)

```

1 CREATE TABLE [dbo].[klasy] (
2     [ID_Klasy] INT IDENTITY (1, 1) NOT NULL,
3     [NumerRoku] INT NOT NULL,
4     [LiteraKlasy] NVARCHAR (1) NOT NULL,
5     [ID_Wychowawcy] INT NULL,
6     CONSTRAINT [PK_Klasy] PRIMARY KEY CLUSTERED ([ID_Klasy] ASC),
7     CONSTRAINT [FK_Klasy_Wychowawcy_ID_Wychowawcy] FOREIGN KEY ([ID_Wychowawcy]) REFERENCES [dbo].[Wychowawcy] (ID_Wychowawcy)
8 );
9
10
11 GO
12 CREATE NONCLUSTERED INDEX [IX_Klasy_ID_Wychowawcy]
13 ON [dbo].[klasy] ([ID_Wychowawcy] ASC);
14
15

```

100 % | Nie znaleziono żadnych problemów | W.: 15 | Zn.: 1 | TABULATORY | MIESZANE

Connection Ready | (localdb)\MSSQLLocalDB | DESKTOP-402\Q49\abagi | Brutus

5. Funkcjonalność aplikacji.

-Administrator tworzy konto dla każdego z użytkowników aplikacji. Tworzy klasę - dodając do niej listę uczniów, przedmiotów oraz wychowawcę. Tylko administrator ma możliwość zmiany danych

pozostałych użytkowników - w tym hasła. W przypadku utraty hasła do konta, odzyskiwania go odbywa się za pośrednictwem maila.

-Jako uczeń mamy możliwość sprawdzania naszych ocen oraz ogłoszeń. Mamy możliwość konwersacji z nauczycielami i przystępowania do testów, po zakończeniu których automatycznie otrzymujemy ocenę, która jest zapisywana. Jako członek klasy jest zapisany do przedmiotów.

-Rodzic ma możliwość konwersacji z nauczycielami oraz wychowawcami. Może przeglądać oceny swoich dzieci oraz może czytać ogłoszenia przesyłane również na mail przez wychowawców.

- Nauczyciel ma możliwość konwersacji z uczniami i rodzicami. Może wstawiać i przeglądać oceny uczniów, z którymi ma zajęcia. Może dodawać i czytać ogłoszenia. Tworzy testy, do których przystępują jego uczniowie.

- Wychowawca – Przypisany do konkretnej klasy ma możliwość wysyłania zbiorowych maili do rodziców uczniów swojej klasy.

- Testy składają się z pytań ABCD.

- Do wiadomości i przedmiotów można dodawać załączniki do 16,7 MB.

6. Interfejs użytkownika.

7. Podsumowanie.

Dodatek A: Skrypty tworzące obiekty baz danych.

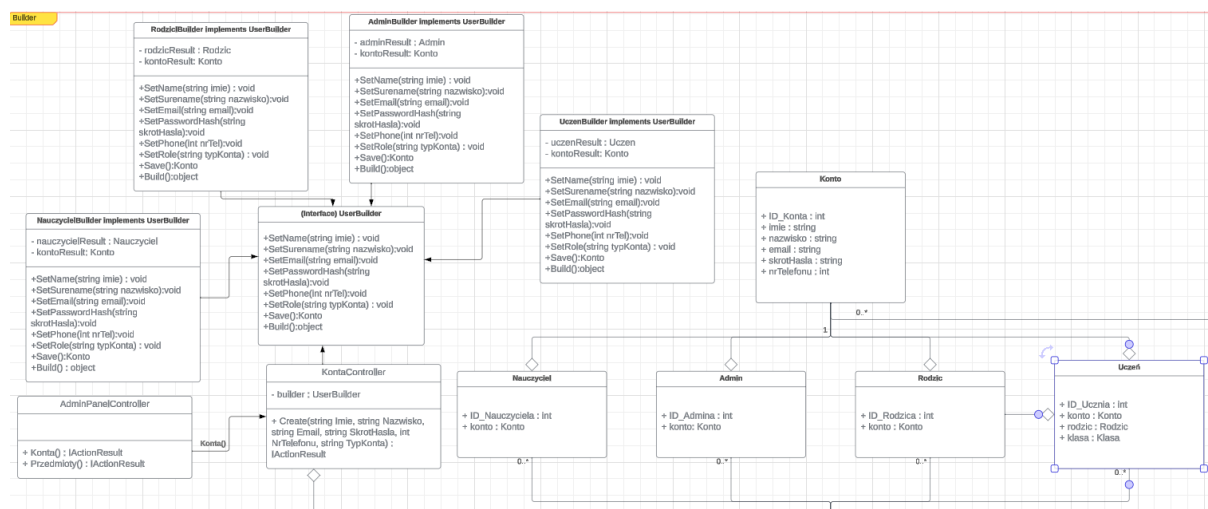
Wirtualny dziennik szkolny – Brutus

Skład:

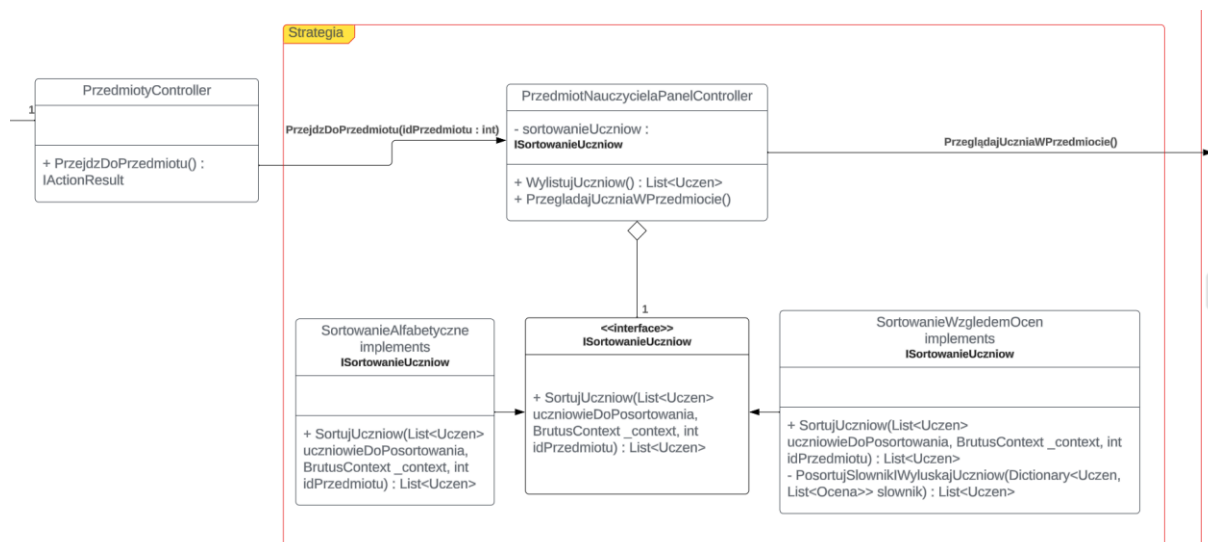
- Dawid Ugniewski
- Adam Bagiński

1. Diagram klas budujących

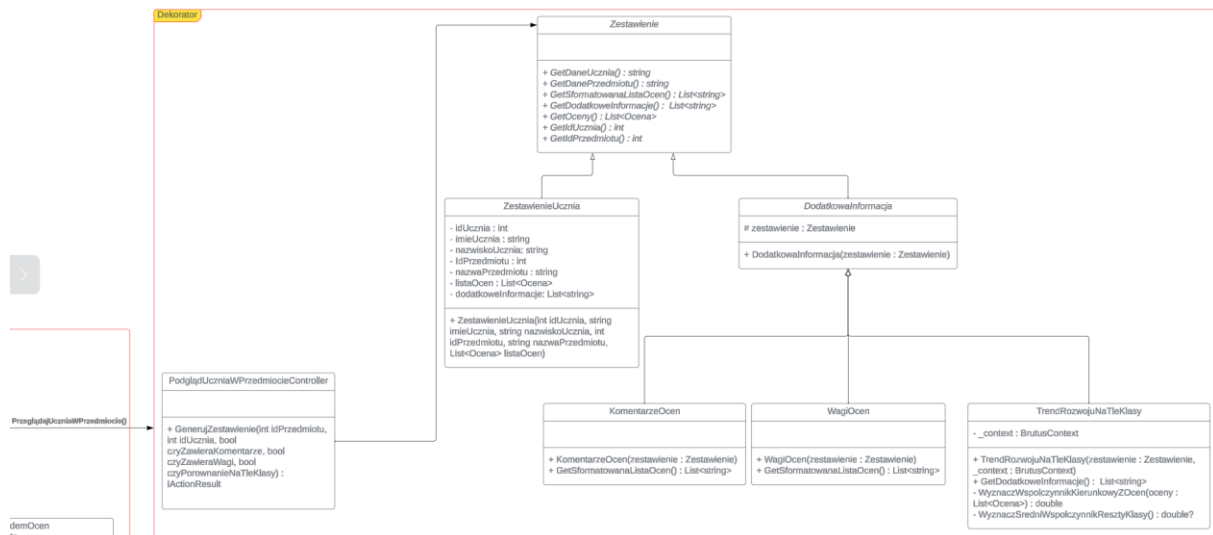
1. Budowniczy (Builder)



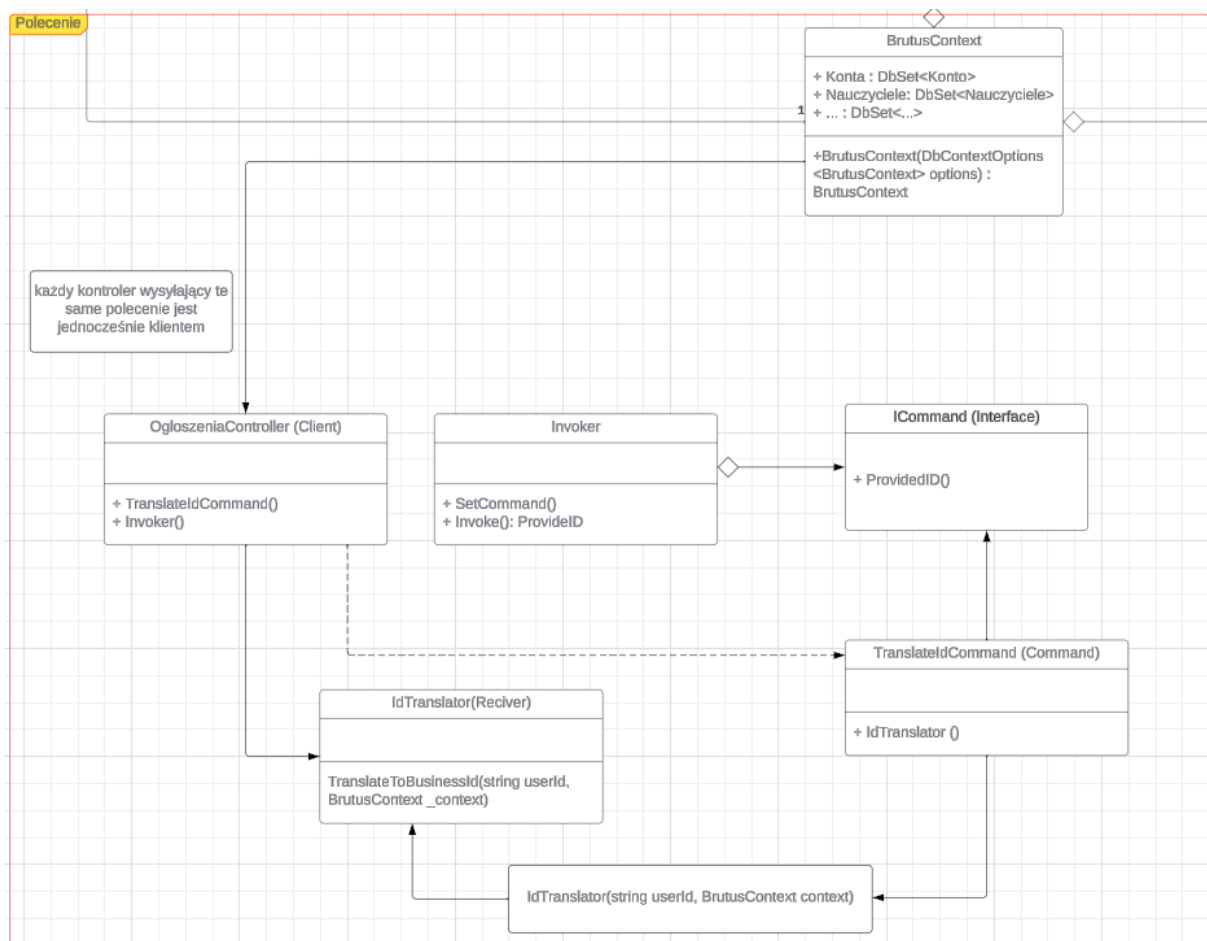
2. Strategia



3. Dekorator



4. Command



2. Opisy użytych wzorców

1. Budowniczy (Builder)

- **Cel użycia:** Wzorec został wykorzystany w procesie tworzenia kont użytkowników. Na podstawie wybranego typu konta aplikacja wybiera odpowiedni builder i tworzy użytkownika, którego zapisuje w bazie danych. Dodatkowo każdy z typów kont ma zmieniany na mail na "szkolny". Do oryginalnego dopisywany jest konkretny typ np. nauczyciel tworzy

konto używając maila abcd@wp.pl - jego mail szkolny będzie wtedy ustawiony jako nauczyciel.abcd@wp.pl.

- **Przyporządkowanie klas do wszystkich ról wzorca:**

Klasy wzorca Budowniczy :

KontaController – Director

UserBuilder – Interfejs

AdminBuilder, NauczycielBuilder, UcenBuilder, RodzicBuilder – konkretne buildery do tworzenia danego typu konta

- **Lokalizacja wzorca Budowniczy w kodzie:**

KontaController (Director) - metoda Create() linie 39 – 84

UserBuilder (Interfejs) - folder /Services/TworzenieUzytkownikow

AdminBuilder, NauczycielBuilder, UcenBuilder, RodzicBuilder (konkretne buildery) - folder /Services/TworzenieUzytkownikow

- **Wektor zmian wzorca Budowniczy (Builder)** - Rozszerzenie o tworzenie kont dla innych użytkowników np. Bibliotekarka (wgląd kto wypożyczył jaką książkę, ile czasu do terminu oddania, dodanie e-karty bibliotecznej), osoba prowadząca projekty finansowane przez UE, sekretariat. Rozszerzenie wymagałoby tylko stworzenie konkretnego buildera i dodanie typu.

2. Strategia

- **Cel użycia:** Nauczyciel prowadzący przedmiot może wyświetlić listę uczniów uczęszczających na przedmiot i może chcieć posortować ich przy użyciu różnych strategii porównania: a) Alfabetycznie b) Względem średniej ocen (najlepszy uczeń -> najgorszy). Podczas przeglądania uczniów nauczyciel może w trakcie działania programu zmieniać sposób sortowania, co w wykonywanej akcji wymagałoby zamieszczenia dużej ilości kodu, a dzięki strategii akcja wyświetlenia uczniów jest przejrzysta, zmiana strategii przez nauczyciela odbywa się w specjalnej metodzie. Łatwo będzie w przyszłości dodać nowe metody sortowania.

- **Przyporządkowanie klas do ról wzorca:**

Kontekst – Controllers\PrzedmiotNauczycielaPanelController.cs

Interface strategii – Services\SortowanieUczniow\ISortowanieUczniow.cs

Strategia alfabetyczna – Services\SortowanieUczniow\SortowanieAlfabetyczne.cs

Strategia względem ocen – Services\SortowanieUczniow\SortowanieWzglememOcen.cs

- **Lokalizacja w kodzie:**

Definicje:

Interface strategii i konkretne strategie - Brutus\Services\SortowanieUczniow\

Kontekst – Brutus\Controllers\PrzedmiotNauczycielaPanelController.cs

Użycie:

Interface strategii - PrzedmiotNauczycielaPanelController.cs - wiersz 15

Użycie konkretnej strategii – PrzedmiotNauczycielaPanelController.cs - wiersze 15 , 74

Metoda do zmiany strategii - PrzedmiotNauczycielaPanelController.cs - wiersze 109-125

- **Wektor zmian:** W przyszłości może zajść potrzeba sortowania uczniów w jeszcze inne sposób. Celem pozostaje posortowana lista uczniów, zmienia się strategia porównywania. Dzięki strategii dużo łatwiej będzie zaimplementować nowe sposoby sortowania.

3. Dekorator

- **Cel użycia:** Nauczyciel prowadzący przedmiot może wyświetlić listę uczniów uczęszczających na przedmiot i dla każdego ucznia wygenerować zestawienie składające się z danych ucznia, przedmiotu, sformatowanej listy ocen i dodatkowych informacji. Zestawienie może być udekorowane w uszczegółowienie informacji o ocenach w liście ocen oraz udekorowane w dodatkowe informacje, np. porównanie trendu rozwoju ucznia (wyliczając regresję liniową) na tle innych na tym przedmiocie. Nauczyciel może dowolnie i w trakcie działania aplikacji wybierać co chce uzyskać w zestawieniu. Dekorowanie zestawienia umożliwia łatwą obsługę różnych żądań nauczyciela i ułatwia rozszerzanie zestawienia w przyszłości.
- **Przyporządkowanie klas do ról wzorca:**

Abstrakcyjny komponent (Zestawienie) - Services\Zestawienie\Zestawienie.cs

Konkretny komponent (ZestawienieUcznia) - Services\Zestawienie\ZestawienieUcznia.cs

Abstrakcyjny dekorator (DodatkowaInformacja) - Services\Zestawienie\DodatkowaInformacja.cs

Konkretny dekorator (WagiOcen) - Services\Zestawienie\WagiOcen.cs

Konkretny dekorator (KomentarzeOcen) - Services\Zestawienie\KomentarzeOcen.cs

Konkretny dekorator (TrendRozwojuNaTleKlasy) - Services\Zestawienie\TrendRozwojuNaTleKlasy.cs

- **Lokalizacja w kodzie:**

Definicje:

Wszystkie klasy wzorca – Brutus\Services\Zestawienie\

Użycie:

W katalogu \Controllers\:

Stworzenie konkretnego zestawienia – PodgladUczniaWPrzedmiocieController.cs wiersze 71-72

Dekorowanie zestawienia – PodgladUczniaWPrzedmiocieController.cs wiersze 76-81

W katalogu \Views\PodgladUczniaWPrzedmiocie\:

Prezentowanie zestawienia nauczycielowi – GenerujZestawienie.cshtml wiersze 1, 6, 7, 16, 32

- **Wektor zmian:** W przyszłości może zajść potrzeba dekorowania zestawienia w nową informację, czy to dotyczącą ocen czy w polu dodatkowych informacji, np. wypisanie dat przy ocenach czy porównanie ucznia na tle tylko najlepszych uczniów w klasie. Zastosowanie dekoratora pozwoli ograniczyć eksplozję klas, która wiązałaby się z naiwnym rozszerzaniem zestawienia przez dziedziczenie.

4. Polecenie (Command)

- **Cel użycia:** Wzorec został wykorzystany do enkapsulacji operacji tłumaczenia identyfikatora użytkownika na identyfikator biznesowy. Posiada on wzorec ICommand z metodą ProvideID służącą wykonaniu polecenia. Klasa Invoker odpowiedzialna za wywołanie operacji zdefiniowanej przez polecenie. Ma ona metodę SetCommand(ICommand command), która pozwala na ustawienie konkretnej komendy do wykonania, oraz metodę Invoke(), która faktycznie wywołuje polecenie poprzez wywołanie ProvideID. Klientami są konkretne kontrolery, które wysyłają polecenie TranslateIdCommand. Odbiorca (Receiver) w tym wypadku to metody IdTranslator oraz TranslateIdCommand. IdTranslator wykonuje pracę związaną ze zmianą identyfikatora.
- **Przyporządkowanie klas do wszystkich ról wzorca:**

Klasy wzorca Polecenie:

ICommand – Interfejs

Invoker - wywołuje metodę wykonawczą

TranslateIdCommand (Command)– polecenie wysyłane przez konkretne kontrolery

IdTranslator - Odbiorca wykonuje pracę związaną ze zmianą identyfikatora

- **Lokalizacja wzorca Budowniczy w kodzie:**

OgloszeniaController (Przykładowe polecenie) 66-70

ICommand(Interfejs) - /Services/Command/ICommand

IdTranslator - /Services/Command/IdTranslator

Invoker - /Services/Command/Invoker

TranslatedCommand - /Services/Command/TranslatedCommand

- **Wektor zmian wzorca Budowniczy (Builder)**

Zmiany ocen – w przypadku poprawy oceny przez ucznia, możliwość jej zmiany przez konkretne polecenie lub dodanie możliwości cofania jej. W przypadku gdy dziennik będzie posiadać sprawdzanie obecności, to gdy jakiś uczeń przyniesie usprawiedliwienie, będzie można cofnąć nieobecność. Wysyłanie powiadomień o zbliżającym się sprawdzaniu, kartkówce lub oddaniu sprawozdania wysyłane za pomocą konkretnego polecenia.

3. Opis co najmniej jednego rozwiązania specyficznego dla użytej technologii i podać wykorzystane źródło wiedzy na ten temat (EK4 - brak tego elementu powoduje niezaliczenie przedmiotu zająć!).

Bardzo często wykorzystywanym (choćby w PrzedmiotNauczycielaPanelController.cs - wiersz 33) w naszej aplikacji rozwiązaniem specyficznym dla użytej technologii było wyrażenie lambda specyficzne dla języka C#.

Wyrażenie lambda konstruuje się poprzez operator =>, po lewej stoją operatory wejściowe, po prawej kod, który wykona się z użyciem tych parametrów.

Przykładem użycia wyrażenia lambda jest ten fragment kodu: “Przedmiot? przedmiot = _context.Przedmioty.FirstOrDefault(p => p.ID_Przedmiotu == idPrzedmiotu);”.

Operator wejściowy to element kolekcji Przedmioty, i na jego podstawie wykonuje się porównanie czy element kolekcji spełnia warunek, dzięki czemu możemy znaleźć w bazie danych odpowiedni przedmiot. Funkcja “p.ID_Przedmiotu == idPrzedmiotu” przyjmuje jako parametr element kolekcji.

Użycie wyrażenia lambda bardzo uprościło pracę z kolekcjami, skomplikowane zapytania LINQ.

Wyrażenie lambda to anonimowa funkcja. Trochę przypomina w działaniu wzorec Polecenie.

Źródło wiedzy: <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/lambda-expressions>

4. Opis podziału pracy w zespole

Dawid Ugniewski:

- Strategia - \Services\SortowanieUczniow\
- Dekorator - \Services\Zestawienie
- Kontrolery - \Controllers\: AccountController.cs, AdminiController.cs, AdminPanelController.cs, HomeController.cs, KlasyController.cs, KontaController.cs (poza metodą Create), NauczycieleController.cs, PodgladUczniaWPrzedmiocieController.cs, PrzedmiotNauczycielaPanelController.cs, PrzedmiotyController.cs
- BrutusContext.cs
- DTOs - \DTOs\ wszystkie
- Models - \Models\: wszystkie
- Views – \Views\: wszystkie widoki dotyczące w.w. kontrolerów.
- Views\Shared_Laout.cshtml
- wwwroot\css\site.css

Adam Bagiński:

- Implementacja wzorca Budowniczy
- Implementacja wzorca Polecenie

Kontrolery:

- /Controllers/WiadomosciController
- /Controllers/UczniowieController
- /Controllers/TestyController
- /Controllers/PytaniaController
- /Controllers/KontraController - Tylko Create() - builder
- /Controllers/OgloszeniaController
- /Controllers/NauczycielPanelController
- /Controllers/RodziceController
- /Controllers/RodzicPanelController

Views:

- /Views/: wszystkie dotyczące w.w. kontrolerów

Services:

- /Services/Command ICommand, IdTranslator, Invoker, TranslatedCommand
- /Services/TworzenieUzytkownikow AdminBuilder, NauczycielBuilder, RodzicBuilder, NauczycielBuilder, UczeńBuilder, UserBuilder

5. Instrukcja użytkownika

- Admin :

- Dodawanie nowego konta | Konta - > Dodaj nowe konto -> Uzupełniamy dane oraz wybieramy rodzaj konta - > Stwórz
- Dodawanie przedmiotu | Przedmioty - > Dodaj nowy przedmiot - > Podajemy jego nazwę i opis np. Nazwa: Matematyka | Opis: Matematyka rozszerzona - > Dodaj
- Przypisywanie nauczyciela do przedmiotu | Przedmioty - > Przypisz nauczyciela - > Wybieramy z listy nauczycieli nauczyciela, który będzie przypisany do przedmiotu jako prowadzący.
- Dodawanie klasy | Klasy - > Dodaj nową klasę - > Podajemy numer oraz literę klasy np. 2C - > Dodaj
- Przypisywanie wychowawcy | Klasy - > Przypisz wychowawcę - > Wybieramy nauczyciela
- Przypisywanie przedmiotów do klasy | Klasy - > Przedmioty klasy - > Wybieramy z listy przedmiot i dodajemy go
- Przypisywanie rodzica do ucznia | Uczniowie - > Ustaw rodzica - > Wybieramy rodzica z listy i przypisujemy go
- Przypisywanie ucznia do klasy | Uczniowie - > Przypisz do klasy - > Wybieramy klasę i przypisujemy
- Możliwość edytowania lub usunięcia kont, klas, przedmiotów etc.

- Uczeń

- Przeglądanie aktualności | Aktualności
- Rozwiązywanie testów | Testy - > Rozpocznij test - > Zaznaczamy odpowiedzi - > Zapisz podejście
//// do poprawy
- Podgląd ocen | Oceny
- Wiadomości

- Nauczyciel

- Przeglądanie i dodawanie ogłoszeń | Ogłoszenia - > Zarządzaj ogłoszeniami - > Wpisujemy treść - > Dodajemy na stronę główną
- Podgląd klasy | Przedmioty - > Przejdź do przedmiotu - > Wylistuj uczniów - > Możemy posortować klasę alfabetycznie lub pod względem ocen - > Podgląd wybranego ucznia
- Tworzenie testów | Testy - > Stwórz test - > Podajemy nazwę, czas trwania oraz ilość pytań - > Możliwość podglądu pytań w teście lub ich dodanie - > Dodaj pytania - > Podajemy treść pytań, warianty odpowiedzi i poprawną odpowiedź

- Wiadomości

//// dodawanie ocen do ucznia

- Rodzic
- Przeglądanie aktualności | Aktualności
- Podgląd ocen swojego dziecka
- Wiadomości

6. Instrukcję instalacji

- Konieczność posiadania odpowiedniego środowiska .NET
- Przywrócenie pakietów za pomocą -dotnet restore
- Uruchamiając aplikację webową na lokalnej bazie danych należy dostosować connectionstring w appsettings.json i zrobić Update-Database dostarczonymi migracjami